

1 Computational Geometry

1.1 convex-hull

```

1 struct node{
2     int x, y;
3     node operator - (const node &b) {
4         return {x - b.x, y - b.y};
5     }
6     int operator * (const node &b) {
7         return x * b.y - y * b.x;
8     }
9     bool operator < (const node &b) {
10         if (x != b.x) return x < b.x;
11         return y < b.y;
12     }
13 };
14 vector<node> build(vector<node> v)
15 {
16     vector<node> ans;
17     sort(v.begin(), v.end());
18     for(int i = 0; i < v.size(); i++){
19         while(ans.size() > 1 and (ans.end()
20             [-2] - ans.end()[-1]) * (ans.end()
21             )[-2] - v[i]) > 0){
22             ans.pop_back();
23         }
24         ans.pb(v[i]);
25     }
26     for(int i = v.size() - 2; i >= 0; i--){
27         while(ans.size() > 1 and (ans.end()
28             [-2] - ans.end()[-1]) * (ans.end()
29             )[-2] - v[i]) > 0){
30             ans.pop_back();
31         }
32     }
33     ans.pb(v[i]);
34     ans.pop_back();
35     return ans;
36 }

```

2 Data Structure

2.1 segment tree lazytag

```

1 struct SegmentTree{
2     struct Node{
3         int sum, f, d, len;
4     } tree[MAXN << 2];
5     void pull(int id){
6         tree[id].sum = tree[id << 1].sum +
7             tree[id << 1 | 1].sum;
8     }
9     void addtag(int id, int a, int d){
10         tree[id].f += a;
11         tree[id].d += d;
12         tree[id].sum += (2 * a + (tree[id].
13             len - 1) * d) * tree[id].len /
14             2;
15     }
16 }

```

```

12 }
13 void push(int id){
14     if(!tree[id].f && !tree[id].d)
15         return;
16     int mid_f = tree[id].f + tree[id <<
17         1].len * tree[id].d;
18     addtag(id << 1, tree[id].f, tree[id
19         ].d);
20     addtag(id << 1 | 1, mid_f, tree[id].
21         d);
22     tree[id].f = tree[id].d = 0;
23 }
24 void build(int l, int r, int id, int ar
25     []){
26     tree[id].len = r - l + 1;
27     tree[id].f = tree[id].d = 0;
28     if(l == r){
29         tree[id].sum = ar[l];
30         return;
31     }
32     int mid = (l + r) >> 1;
33     build(l, mid, id << 1, ar);
34     build(mid + 1, r, id << 1 | 1, ar);
35     pull(id);
36 }
37 void update(int ql, int qr, int a, int d
38     , int l, int r, int id){
39     if(ql <= l and r <= qr){
40         addtag(id, a + (l - ql) * d, d);
41         return;
42     }
43     push(id);
44     int mid = (l + r) >> 1;
45     if(ql <= mid) update(ql, qr, a, d, l
46         , mid, id << 1);
47     if(qr > mid) update(ql, qr, a, d,
48         mid + 1, r, id << 1 | 1);
49     pull(id);
50 }
51 int query(int ql, int qr, int l, int r,
52     int id){
53     if(ql <= l and r <= qr) return tree[
54         id].sum;
55     push(id);
56     int mid = (l + r) >> 1;
57     if(ql <= mid) re += query(ql, qr, l,
58         mid, id << 1);
59     if(qr > mid) re += query(ql, qr, mid
60         + 1, r, id << 1 | 1);
61     return re;
62 }
63 }
64 }

```

2.2 treap

```

1 #include<bits/stdc++.h>
2 using namespace std;
3 #define KCC ios::sync_with_stdio(0);cin.tie
4 (0);
5 #define pb push_back
6 #define pii pair<int, int>
7 #define F first
8 #define S second

```

```

8
9 mt19937 rnd(time(0));
10 const int MAXN = 2e5 + 5;
11 int left_node[MAXN], right_node[MAXN],
12     priority[MAXN], sz[MAXN], node_cnt = 0,
13     root = 0;
14 char val[MAXN];
15 void up(int i)
16 {
17     sz[i] = sz[left_node[i]] + sz[right_node
18         [i]] + 1;
19 }
20 void split(int i, int l, int r, int rank)
21 {
22     if(i == 0){
23         left_node[r] = right_node[l] = 0;
24         return;
25     }
26     if(sz[left_node[i]] + 1 <= rank){
27         right_node[l] = i;
28         split(right_node[i], i, r, rank - sz
29             [left_node[i]] - 1);
30     }
31     else{
32         left_node[r] = i;
33         split(left_node[i], l, i, rank);
34     }
35     up(i);
36 }
37 int merge(int l, int r)
38 {
39     if(l == 0 or r == 0) return l + r;
40     if(priority[l] >= priority[r]){
41         right_node[l] = merge(right_node[l],
42             r);
43         up(l);
44         return l;
45     }
46     else{
47         left_node[r] = merge(l, left_node[r
48             ]);
49         up(r);
50         return r;
51     }
52 }
53 void out(int i)
54 {
55     if(i == 0) return;
56     out(left_node[i]);
57     cout << val[i];
58     out(right_node[i]);
59 }
60 int main()
61 {
62     KCC
63     int n, m;
64     cin >> n >> m;
65     for(int i = 0; i < n; i++){
66         char c;
67         cin >> c;
68         priority[++node_cnt] = rnd();
69         val[node_cnt] = c;
70         sz[node_cnt] = 1;
71         root = merge(root, node_cnt);
72     }
73 }

```

```

68 }
69 while(m--){
70     int a, b;
71     cin >> a >> b;
72
73     split(root, 0, 0, b);
74     int lm = right_node[0];
75     int r = left_node[0];
76
77     split(lm, 0, 0, a - 1);
78     int l = right_node[0];
79     int m = left_node[0];
80
81     root = merge(merge(l, r), m);
82 }
83 out(root);
84 return 0;
85 }

```

3 Flow

3.1 dinic

```

1 struct Dinic{
2     struct Edge{
3         int to, cap, rev;
4     };
5     vector<Edge> adj[505];
6     vector<int> level, ptr;
7     int n;
8     Dinic(int _n){
9         n = _n;
10    }
11    void addedge(int u, int v, int w){
12        adj[u].pb({v, w, adj[v].size()});
13        adj[v].pb({u, 0, adj[u].size() - 1});
14    }
15    bool bfs(int s, int t)
16    {
17        fill(level.begin(), level.end(), -1)
18        ;
19        level[s] = 0;
20        queue<int> q;
21        q.push(s);
22        while(q.size()){
23            int x = q.front(); q.pop();
24            for(auto edge : adj[x]){
25                if(edge.cap > 0 and level[
26                    edge.to] == -1){
27                    level[edge.to] = level[x
28                        ] + 1;
29                    q.push(edge.to);
30                }
31            }
32        }
33        return (level[t] != -1);
34    }
35    int dfs(int now, int t, int pushed)
36    {
37        if(now == t) return pushed;
38    }
39 }

```

```

35     for(int &i = ptr[now]; i < adj[now].
        size(); i++){
36         auto &w = adj[now][i];
37         if(level[now] + 1 != level[w.to]
            or w.cap == 0) continue;
38         int tr = dfs(w.to, t, min(pushed
            , w.cap));
39         if(tr == 0) continue;
40         w.cap -= tr;
41         adj[w.to][w.rev].cap += tr;
42         return tr;
43     }
44     return 0;
45 }
46 int compute_max_flow(){
47     int max_flow = 0;
48     while(bfs(1, n)){
49         fill(ptr.begin(), ptr.end(), 0);
50         while(int pushed = dfs(1, n, 1
            e18)){
51             max_flow += pushed;
52         }
53     }
54     return max_flow;
55 }
56 };

```

4 Graph

4.1 bellman-ford

```

1 void bellman_ford(){
2     for(int i = 1; i <= n; i++){
3         bool update = false;
4         for(int j = 1; j <= n; j++){
5             for(auto k : g[j]){
6                 if(dis[k.F] > dis[j] + k.S)
7                     update = true, dis[k.F]
                        = dis[j] + k.S;
8             }
9             if(!update)break;
10            if(i == n)cout << "neg_cycle" << "\n";
11        }
12    }
13 }

```

4.2 SCC

```

1 struct SCC{
2     vector<vector<int>> g, rg;
3     vector<bool> vis, used;
4     vector<int> component, st;
5     int component_cnt = 0, n;
6     void dfs1(int node){
7         vis[node] = true;
8         for(int i : g[node]) if(!vis[i])
9             dfs1(i);
10        st.pb(node);
11    }
12 }

```

```

11 void dfs2(int node){
12     used[node] = true;
13     for(int i : rg[node]) if(!used[i])
14         dfs2(i);
15     component[node] = component_cnt;
16 }
17 SCC(const vector<vector<int>> v){
18     n = v.size() - 1;
19     g = v;
20     rg.resize(n + 1);
21     vis.resize(n + 1, false);
22     used.resize(n + 1, false);
23     component.resize(n + 1, 0);
24     for(int i = 1; i <= n; i++) for(int
        j : v[i]) rg[j].pb(i);
25 }
26 vector<int> find_component(){
27     for(int i = 1; i <= n; i++) if(!vis[
        i]) dfs1(i);
28     reverse(st.begin(), st.end());
29     for(int i : st) if(!used[i]){
30         component_cnt++;
31         dfs2(i);
32     }
33     return component;
34 }

```

5 Linear Programming

5.1 simplex

```

1 /*target:
2     max \sum_{j=1}^n A_{0,j} * x_j
3 condition:
4     \sum_{j=1}^n A_{i,j} * x_j <= A_{i,0} / i=1~m
5     x_j >= 0 / j=1~n
6 VDB = vector<double>*/
7 template<class VDB>
8 VDB simplex(int m,int n,vector<VDB> a){
9     vector<int> left(m+1), up(n+1);
10    iota(left.begin(), left.end(), n);
11    iota(up.begin(), up.end(), 0);
12    auto pivot = [&](int x, int y){
13        swap(left[x], up[y]);
14        auto k = a[x][y]; a[x][y] = 1;
15        vector<int> pos;
16        for(int j = 0; j <= n; j++){
17            a[x][j] /= k;
18            if(a[x][j] != 0) pos.push_back(j);
19        }
20        for(int i = 0; i <= m; i++){
21            if(a[i][y]==0 || i == x) continue;
22            k = a[i][y], a[i][y] = 0;
23            for(int j : pos) a[i][j] -= k*a[x][j];
24        }
25    };
26    for(int x,y;;){
27        for(int i=x+1; i <= m; ++i)
28            if(a[i][0]<a[x][0]) x = i;
29        if(a[x][0]>=0) break;
30    }
31 }

```

```

30     for(int j=y+1; j <= n; ++j)
31         if(a[x][j]<a[x][y]) y = j;
32     if(a[x][y]>=0) return VDB(); //infeasible
33     pivot(x, y);
34 }
35 for(int x,y;;){
36     for(int j=y+1; j <= n; ++j)
37         if(a[0][j] > a[0][y]) y = j;
38     if(a[0][y]<=0) break;
39     x = -1;
40     for(int i=1; i<=m; ++i) if(a[i][y] > 0)
41         if(x == -1 || a[i][0]/a[i][y]
42             < a[x][0]/a[x][y]) x = i;
43     if(x == -1) return VDB(); //unbounded
44     pivot(x, y);
45 }
46 VDB ans(n + 1);
47 for(int i = 1; i <= m; ++i)
48     if(left[i] <= n) ans[left[i]] = a[i][0];
49 ans[0] = -a[0][0];
50 return ans;
51 }

```

6 Number Theory

6.1 basic

```

1 template<typename T>
2 void gcd(const T &a,const T &b,T &d,T &x,T &y){
3     if(!b) d=a,x=1,y=0;
4     else gcd(b,a%b,d,y,x), y-=x*(a/b);
5 }
6 long long int phi[N+1];
7 void phiTable(){
8     for(int i=1;i<=N;i++)phi[i]=i;
9     for(int i=1;i<=N;i++)for(x=i*2;x<=N;x+=i)
10        phi[x]-=phi[i];
11 }
12 void all_divdown(const LL &n){ // all n/x
13     for(LL a=1;a<=n;a=n/(n/(a+1))){
14         // dosomething;
15     }
16 }
17 const int MAXPRIME = 1000000;
18 int iscom[MAXPRIME], prime[MAXPRIME],
    primecnt;
19 int phi[MAXPRIME], mu[MAXPRIME];
20 void sieve(void){
21     memset(iscom,0,sizeof(iscom));
22     primecnt = 0;
23     phi[1] = mu[1] = 1;
24     for(int i=2;i<MAXPRIME;++i) {
25         if(!iscom[i]) {
26             prime[primecnt++] = i;
27             mu[i] = -1;
28             phi[i] = i-1;
29         }
30         for(int j=0;j<primecnt;++j) {
31             int k = i * prime[j];
32             if(k>=MAXPRIME) break;
33         }
34     }
35 }

```

```

32     iscom[k] = prime[j];
33     if(i%prime[j]==0) {
34         mu[k] = 0;
35         phi[k] = phi[i] * prime[j];
36         break;
37     } else {
38         mu[k] = -mu[i];
39         phi[k] = phi[i] * (prime[j]-1);
40     }
41 }
42 }
43 }
44 }
45 bool g_test(const LL &g, const LL &p, const
    vector<LL> &v) {
46     for(int i=0;i<v.size();++i)
47         if(modexp(g,(p-1)/v[i],p)==1)
48             return false;
49     return true;
50 }
51 LL primitive_root(const LL &p) {
52     if(p==2) return 1;
53     vector<LL> v;
54     Factor(p-1,v);
55     v.erase(unique(v.begin(), v.end()), v.end
        ());
56     for(LL g=2;g<p;++g)
57         if(g_test(g,p,v))
58             return g;
59     puts("primitive_root NOT FOUND");
60     return -1;
61 }
62 int Legendre(const LL &a, const LL &p) {
63     return modexp(a%p,(p-1)/2,p); }
64 LL inv(const LL &a, const LL &n) {
65     LL d,x,y;
66     gcd(a,n,d,x,y);
67     return d==1 ? (x+n)%n : -1;
68 }
69 }
70 int inv[maxN];
71 LL invtable(int n,LL P){
72     inv[1]=1;
73     for(int i=2;i<n;++i)
74         inv[i]=(P-(P/i))*inv[P%i]%P;
75 }
76 }
77 LL log_mod(const LL &a, const LL &b, const
    LL &p) {
78     // a ^ x = b ( mod p )
79     int m=sqrt(p+.5), e=1;
80     LL v=inv(modexp(a,m,p), p);
81     map<LL,int> x;
82     x[1]=0;
83     for(int i=1;i<m;++i) {
84         e = LLMul(e,a,p);
85         if(!x.count(e)) x[e] = i;
86     }
87     for(int i=0;i<m;++i) {
88         if(x.count(b)) return i*m + x[b];
89         b = LLMul(b,v,p);
90     }
91     return -1;
92 }
93 }

```

```

94 LL Tonelli_Shanks(const LL &n, const LL &p)
95 {
96     //  $x^2 = n \pmod p$ 
97     if(n==0) return 0;
98     if(Legendre(n,p)!=1) while(1) { puts("SQR
99         ROOT does not exist"); }
100     int S = 0;
101     LL Q = p-1;
102     while( !(Q&1) ) { Q>>=1; ++S; }
103     if(S==1) return modexp(n%p,(p+1)/4,p);
104     LL z = 2;
105     for(; Legendre(z,p)!=-1; ++z)
106         LL c = modexp(z,Q,p);
107     LL R = modexp(n%p,(Q+1)/2,p), t = modexp(n
108         %p,Q,p);
109     int M = S;
110     while(1) {
111         if(t==1) return R;
112         LL b = modexp(c,1L<<(M-i-1),p);
113         R = LLMul(R,b,p);
114         t = LLMul( LLMul(b,b,p), t, p);
115         c = LLMul(b,b,p);
116         M = i;
117     }
118     return -1;
119 }
120 template<typename T>
121 T Euler(T n){
122     T ans=n;
123     for(T i=2;i*i<=n;++i){
124         if(n%i==0){
125             ans=ans/i*(i-1);
126             while(n%i==0)n/=i;
127         }
128     }
129     //Chinese_remainder_theorem
130     template<typename T>
131     T pow_mod(T n,T k,T m){
132         T ans=1;
133         for(n=(n>m?n%m:n);k>>=1){
134             if(k&1)ans=ans*n%m;
135             n=n*n%m;
136         }
137         return ans;
138     }
139     template<typename T>
140     T crt(vector<T> &m,vector<T> &a){
141         T M=1,tM,ans=0;
142         for(int i=0;i<(int)m.size();++i)M*=m[i];
143         for(int i=0;i<(int)a.size();++i){
144             tM=M/m[i];
145             ans=(ans+(a[i]*tM%M)*pow_mod(tM,Euler(m[i])-1,m[i])%M)%M;
146         }
147         //如果m[i]是質數·Euler(m[i])-1=m[i]-2·
148         //就不用算Euler了*/
149     }
150     return ans;
151 }
152 //java code
153

```

```

154 //求sqrt(N)的連分數
155 public static void Pell(int n){
156     BigInteger N,p1,p2,q1,q2,a0,a1,a2,g1,g2,h1
157         ,h2,p,q;
158     g1=q2=p1=BigInteger.ZERO;
159     h1=q1=p2=BigInteger.ONE;
160     a0=a1=BigInteger.valueOf((int)Math.sqrt
161         (1.0*n));
162     BigInteger ans=a0.multiply(a0);
163     if(ans.equals(BigInteger.valueOf(n))){
164         System.out.println("No solution!");
165         return ;
166     }
167     while(true){
168         g2=a1.multiply(h1).subtract(g1);
169         h2=N.subtract(g2.pow(2)).divide(h1);
170         a2=g2.add(a0).divide(h2);
171         p=a1.multiply(p2).add(p1);
172         q=a1.multiply(q2).add(q1);
173         if(p.pow(2).subtract(N.multiply(q.pow
174             (2))).compareTo(BigInteger.ONE)==0)
175             break;
176         g1=g2;h1=h2;a1=a2;
177         p1=p2;p2=p;
178         q1=q2;q2=q;
179     }
180     System.out.println(p+" "+q);
181 }

```

6.2 bit set

```

1 void sub_set(int S){
2     int sub=S;
3     do{
4         //對某集合的子集合的處理
5         sub=(sub-1)&S;
6     }while(sub!=S);
7 }
8 void k_sub_set(int k,int n){
9     int comb=(1<<k)-1,S=1<<n;
10    while(comb<S){
11        //對大小為k的子集合的處理
12        int x=comb&-comb,y=comb+x;
13        comb=((comb&y)/x>>1)|y;
14    }
15 }

```

6.3 cantor expansion

```

1 int factorial[MAXN];
2 void init(){
3     factorial[0]=1;
4     for(int i=1;i<=MAXN;++i)factorial[i]=
5         factorial[i-1]*i;
6 }
7 int encode(const vector<int> &s){
8     int n=s.size(),res=0;
9     for(int i=0;i<n;++i){
10        int t=0;

```

```

10     for(int j=i+1;j<n;++j)
11         if(s[j]<s[i])++t;
12     res+=t*factorial[n-i-1];
13 }
14 return res;
15 }
16 vector<int> decode(int a,int n){
17     vector<int> res;
18     vector<bool> vis(n,0);
19     for(int i=n-1;i>=0;--i){
20         int t=a/factorial[i],j;
21         for(j=0;j<n;++j)
22             if(!vis[j]){
23                 if(t==0)break;
24                 --t;
25             }
26         res.push_back(j);
27         vis[j]=1;
28         a%=factorial[i];
29     }
30     return res;
31 }

```

6.4 FFT

```

1 template<typename T,typename VT=vector<
2     complex<T> >>
3 struct FFT{
4     const T pi;
5     FFT(const T pi=acos((T)-1)):pi(pi){}
6     unsigned bit_reverse(unsigned a,int len){
7         a=((a&0x55555555U)<<1)|((a&0xAAAAAAAAU)>>1);
8         a=((a&0x33333333U)<<2)|((a&0xCCCCCCCCU)>>2);
9         a=((a&0x0F0F0F0FU)<<4)|((a&0xF0F0F0FU)>>4);
10        a=((a&0x00FF00FFU)<<8)|((a&0xFF00FF00U)>>8);
11        a=((a&0x0000FFFFU)<<16)|((a&0xFFFF0000U)
12            >>16);
13        return a>>(32-len);
14    }
15    void fft(bool is_inv,VT &in,VT &out,int N)
16    {
17        int bitlen=__lg(N),num=is_inv?-1:1;
18        for(int i=0;i<N;++i)out[bit_reverse(i,
19            bitlen)]=in[i];
20        for(int step=2;step<=N;step<=1){
21            const int mh=step>>1;
22            for(int i=0;i<mh;++i){
23                complex<T> wi=exp(complex<T>(0,i*num
24                    *pi/mh));
25                for(int j=i;j<N;j+=step){
26                    int k=j+mh;
27                    complex<T> u=out[j],t=wi*out[k];
28                    out[j]=u+t;
29                    out[k]=u-t;
30                }
31            }
32        }
33        if(is_inv)for(int i=0;i<N;++i)out[i]/=N;
34    }
35 };

```

6.5 find real root

```

1 //  $an^x + \dots + a_1x + a_0 = 0$ ;
2 int sign(double x){
3     return x < -eps ? -1 : x > eps;
4 }
5
6 double get(const vector<double>&coef, double
7     x){
8     double e = 1, s = 0;
9     for(auto i : coef) s += i*e, e *= x;
10    return s;
11 }
12 double find(const vector<double>&coef, int n
13     , double lo, double hi){
14     double sign_lo, sign_hi;
15     if( !(sign_lo = sign(get(coef,lo))) )
16         return lo;
17     if( !(sign_hi = sign(get(coef,hi))) )
18         return hi;
19     if(sign_lo * sign_hi > 0) return INF;
20     for(int stp = 0; stp < 100 && hi - lo >
21         eps; ++stp){
22         double m = (lo+hi)/2.0;
23         int sign_mid = sign(get(coef,m));
24         if(!sign_mid) return m;
25         if(sign_lo*sign_mid < 0) hi = m;
26         else lo = m;
27     }
28     return (lo+hi)/2.0;
29 }
30 vector<double> cal(vector<double>coef, int n
31     ){
32     vector<double>res;
33     if(n == 1){
34         if(sign(coef[1]) res.pb(-coef[0]/coef
35             [1]);
36         return res;
37     }
38     vector<double>dcoef(n);
39     for(int i = 0; i < n; ++i) dcoef[i] = coef
40         [i+1]*(i+1);
41     vector<double>droot = cal(dcoef, n-1);
42     droot.insert(droot.begin(), -INF);
43     droot.pb(INF);
44     for(int i = 0; i+1 < droot.size(); ++i){
45         double tmp = find(coef, n, droot[i],
46             droot[i+1]);
47         if(tmp < INF) res.pb(tmp);
48     }
49     return res;
50 }
51 int main () {
52     vector<double>ve;
53     vector<double>ans = cal(ve, n);
54     // 視情況把答案 +eps · 避免 -0
55 }

```

6.6 FWT

```

1 vector<int> F_OR_T(vector<int> f, bool
  inverse){
2   for(int i=0; (2<<i)<=f.size(); ++i)
3     for(int j=0; j<f.size(); j+=2<<i)
4       for(int k=0; k<(1<<i); ++k)
5         f[j+k+(1<<i)] += f[j+k]*(inverse
6           ?-1:1);
7   return f;
8 }
9 vector<int> rev(vector<int> A) {
10   for(int i=0; i<A.size(); i+=2)
11     swap(A[i],A[i^(A.size()-1)]);
12   return A;
13 }
14 vector<int> F_AND_T(vector<int> f, bool
  inverse){
15   return rev(F_OR_T(rev(f), inverse));
16 }
17 vector<int> F_XOR_T(vector<int> f, bool
  inverse){
18   for(int i=0; (2<<i)<=f.size(); ++i)
19     for(int j=0; j<f.size(); j+=2<<i)
20       for(int k=0; k<(1<<i); ++k){
21         int u=f[j+k], v=f[j+k+(1<<i)];
22         f[j+k+(1<<i)] = u-v, f[j+k] = u+v;
23       }
24   if(inverse) for(auto &a:f) a/=f.size();
25   return f;
26 }

```

6.7 LinearCongruence

```

1 pair<LL,LL> LinearCongruence(LL a[],LL b[],
  LL m[],int n) {
2   // a[i]*x = b[i] ( mod m[i] )
3   for(int i=0;i<n;++i) {
4     LL x, y, d = extgcd(a[i],m[i],x,y);
5     if(b[i]%d!=0) return make_pair(-1LL,0LL)
6     ;
7     m[i] /= d;
8     b[i] = LLmul(b[i]/d,x,m[i]);
9   }
10  LL lastb = b[0], lastm = m[0];
11  for(int i=1;i<n;++i) {
12    LL x, y, d = extgcd(m[i],lastm,x,y);
13    if((lastb-b[i])%d!=0) return make_pair
14      (-1LL,0LL);
15    lastb = LLmul((lastb-b[i])/d,x,(lastm/d)
16      )*m[i];
17    lastm = (lastm/d)*m[i];
18    lastb = (lastb+b[i])%lastm;
19  }
20  return make_pair(lastb<0?lastb+lastm:lastb
21    ,lastm);
22 }

```

6.8 Lucas

```

1 ll C(ll n, ll m, ll p){ // n!/m!/(n-m)!
2   if(n<m) return 0;

```

```

3   return f[n]*inv(f[m],p)%inv(f[n-m],p)%p;
4 }
5 ll L(ll n, ll m, ll p){
6   if(!m) return 1;
7   return C(n%p,m%p,p)*L(n/p,m/p,p)%p;
8 }
9 ll Wilson(ll n, ll p){ // n!%p
10  if(!n)return 1;
11  ll res=Wilson(n/p, p);
12  if((n/p)%2) return res*(p-f[n%p])%p;
13  return res*f[n%p]%p; //(p-1)!%p=-1
14 }

```

6.9 Matrix

```

1 template<typename T>
2 struct Matrix{
3   using rt = std::vector<T>;
4   using mt = std::vector<rt>;
5   using matrix = Matrix<T>;
6   int r,c;
7   mt m;
8   Matrix(int r,int c):r(r),c(c),m(r,rt(c)){}
9   rt& operator[](int i){return m[i];}
10  matrix operator+(const matrix &a){
11    matrix rev(r,c);
12    for(int i=0;i<r;++i)
13      for(int j=0;j<c;++j)
14        rev[i][j]=m[i][j]+a.m[i][j];
15    return rev;
16  }
17  matrix operator-(const matrix &a){
18    matrix rev(r,c);
19    for(int i=0;i<r;++i)
20      for(int j=0;j<c;++j)
21        rev[i][j]=m[i][j]-a.m[i][j];
22    return rev;
23  }
24  matrix operator*(const matrix &a){
25    matrix rev(r,a.c);
26    matrix tmp(a.c,a.r);
27    for(int i=0;i<a.r;++i)
28      for(int j=0;j<a.c;++j)
29        tmp[j][i]=a.m[i][j];
30    for(int i=0;i<r;++i)
31      for(int j=0;j<a.c;++j)
32        for(int k=0;k<c;++k)
33          rev.m[i][j]+=m[i][k]*tmp[j][k];
34    return rev;
35  }
36  bool inverse(){
37    Matrix t(r,r+c);
38    for(int y=0;y<r;y++){
39      t.m[y][c+y] = 1;
40      for(int x=0;x<c;++x)
41        t.m[y][x]=m[y][x];
42    }
43    if( !t.gas() )
44      return false;
45    for(int y=0;y<r;y++){
46      for(int x=0;x<c;++x)
47        m[y][x]=t.m[y][c+x]/t.m[y][y];
48    return true;

```

```

49  }
50  T gas(){
51    vector<T> lazy(r,1);
52    bool sign=false;
53    for(int i=0;i<r;++i){
54      if( m[i][i]==0 ){
55        int j=i+1;
56        while(j<r&&!m[j][i])j++;
57        if(j==r)continue;
58        m[i].swap(m[j]);
59        sign=!sign;
60      }
61      for(int j=0;j<r;j++){
62        if(i==j)continue;
63        lazy[j]=lazy[j]*m[i][i];
64        T mx=m[j][i];
65        for(int k=0;k<c;k++){
66          m[j][k]=m[j][k]*m[i][i]-m[i][k]*mx
67            ;
68        }
69      }
70      T det=sign?-1:1;
71      for(int i=0;i<r;++i){
72        det = det*m[i][i];
73        det = det/lazy[i];
74      }
75      return det;
76    }
77  };

```

6.10 MillerRobin

```

1 ULL LLmul(ULL a, ULL b, const ULL &mod) {
2   LL ans=0;
3   while(b) {
4     if(b&1) {
5       ans+=a;
6       if(ans>=mod) ans-=mod;
7     }
8     a<<=1, b>>=1;
9     if(a>=mod) a-=mod;
10  }
11  return ans;
12 }
13 ULL mod_mul(ULL a,ULL b,ULL m){
14  a%=m,b%=m; /* fast for m < 2^58 */
15  ULL y=(ULL)((double)a*b/m+0.5);
16  ULL r=(a*b-y*m)%m;
17  return r<0?r+m:r;
18 }
19 template<typename T>
20 T pow(T a,T b,T mod){ //a^b%mod
21  T ans=1;
22  for(;b;a=mod_mul(a,a,mod),b>>=1)
23    if(b&1)ans=mod_mul(ans,a,mod);
24  return ans;
25 }
26 int sprp[3]={2,7,61}; //int範圍可解
27 int llsp[7]={2,325,9375,28178,450775,9780504,
28 1795265022}; //至少 unsigned long long範圍

```

```

29 template<typename T>
30 bool isprime(T n,int *sprp,int num){
31   if(n==2)return 1;
32   if(n<2||n%2==0)return 0;
33   int t=0;
34   T u=n-1;
35   for(;u%2==0;++t)u>>=1;
36   for(int i=0;i<num;++i){
37     T a=sprp[i]%n;
38     if(a==0||a==1||a==n-1)continue;
39     T x=pow(a,u,n);
40     if(x==1||x==n-1)continue;
41     for(int j=0;j<t;j++){
42       x=mod_mul(x,x,n);
43       if(x==1)return 0;
44       if(x==n-1)break;
45     }
46     if(x==n-1)continue;
47     return 0;
48   }
49   return 1;
50 }

```

6.11 NTT

```

1 2615053605667*(2^18)+1,3
2 15*(2^27)+1,31
3 479*(2^21)+1,3
4 7*17*(2^23)+1,3
5 3*3*211*(2^19)+1,5
6 25*(2^22)+1,3
7 template<typename T,typename VT=vector<T> >
8 struct NTT{
9   const T P,G;
10  NTT(T p=(1<<23)*7*17+1,T g=3):P(p),G(g){}
11  unsigned bit_reverse(unsigned a,int len){
12    //Look FFT.cpp
13  }
14  T pow_mod(T n,T k,T m){
15    T ans=1;
16    for(n=(n>=m?n%m:n);k>>=1){
17      if(k&1)ans=ans*n%m;
18      n=n*n%m;
19    }
20    return ans;
21  }
22  void ntt(bool is_inv,VT &in,VT &out,int N)
23  {
24    int bitlen=__lg(N);
25    for(int i=0;i<N;++i)out[bit_reverse(i,
26      bitlen)]=in[i];
27    for(int step=2,id=1;step<=N;step<=1,++
28      id){
29      T wn=pow_mod(G,(P-1)>>id,P),wi=1,u,t;
30      for(int i=0;i<N;i+=step){
31        const int mh=step>>1;
32        for(int j=i;j<N;j+=step){
33          u=out[j],t=wi*out[j+mh]%P;
34          out[j]=u+t;
35          out[j+mh]=u-t;
36          if(out[j]>=P)out[j]-=P;
37          if(out[j+mh]<0)out[j+mh]+=P;
38        }
39        wi=wn;
40      }
41      if(is_inv) for(int i=0;i<N;i++)out[i]=
42        pow(out[i],P-2,P);
43    }
44  }

```

```

36         wi=wi*wn%P;
37     }
38 }
39 if(is_inv){
40     for(int i=1;i<N/2;++i)swap(out[i],out[
41         N-i]);
42     T invn=pow_mod(N,P-2,P);
43     for(int i=0;i<N;++i)out[i]=out[i]*invn
44         %P;
45 }

```

6.12 Simpson

```

1 double simpson(double a,double b){
2     double c=a+(b-a)/2;
3     return (F(a)+4*F(c)+F(b))*(b-a)/6;
4 }
5 double asr(double a,double b,double eps,
6     double A){
7     double c=a+(b-a)/2;
8     double L=simpson(a,c),R=simpson(c,b);
9     if( abs(L+R-A)<15*eps )
10         return L+R+(L+R-A)/15.0;
11     return asr(a,c,eps/2,L)+asr(c,b,eps/2,R);
12 }
13 double asr(double a,double b,double eps){
14     return asr(a,b,eps,simpson(a,b));
15 }

```

6.13 外星模運算

```

1 //a[0]^(a[1]^a[2]^...)
2 #define maxn 1000000
3 int euler[maxn+5];
4 bool is_prime[maxn+5];
5 void init_euler(){
6     is_prime[1]=1; // 1 不是質數
7     for(int i=1;i<=maxn;i++)euler[i]=i;
8     for(int i=2;i<=maxn;i++){
9         if(!is_prime[i]){ // 是質數
10             euler[i]--;
11             for(int j=i<1;j<=maxn;j+=i){
12                 is_prime[j]=1;
13                 euler[j]=euler[j]/i*(i-1);
14             }
15         }
16     }
17 }
18 LL pow(LL a,LL b,LL mod){ // a^b%mod
19     LL ans=1;
20     for(;b;a=a*a%mod,b>=1)
21         if(b&1)ans=ans*a%mod;
22     return ans;
23 }
24 bool isless(LL *a,int n,int k){
25     if(*a==1)return k>1;
26     if(--n==0)return *a<k;

```

```

27 int next=0;
28 for(LL b=1;b<k;++next)
29     b*=a;
30 return isless(a+1,n,next);
31 }
32 LL high_pow(LL *a,int n,LL mod){
33     if(*a==1||--n==0)return *a%mod;
34     int k=0,r=euler[mod];
35     for(LL tma=1;tma!=pow(*a,k+r,mod);++k)
36         tma=tma*(a%mod);
37     if(isless(a+1,n,k))return pow(*a,high_pow(
38         a+1,n,k),mod);
39     int tmd=high_pow(a+1,n,r), t=(tmd-k+r)%r;
40     return pow(*a,k+t,mod);
41 }
42 LL a[1000005];
43 int t,mod;
44 int main(){
45     init_euler();
46     scanf("%d",&t);
47     #define n 4
48     while(t--){
49         for(int i=0;i<n;++i)scanf("%Lld",&a[i]);
50         scanf("%d",&mod);
51         printf("%Lld\n",high_pow(a,n,mod));
52     }
53 }

```

6.14 數位統計

```

1 ll d[65], dp[65][2]; // up 區間是不是完整
2 ll dfs(int p,bool is8,bool up){
3     if(!p)return 1; // 回傳0 是不是答案
4     if(!up&&~dp[p][is8])return dp[p][is8];
5     int mx = up?d[p]:9; // 可以用的有那些
6     ll ans=0;
7     for(int i=0;i<=mx;++i){
8         if( is8&&i==7 )continue;
9         ans += dfs(p-1,i==8,up&&i==mx);
10    }
11    if(!up)dp[p][is8]=ans;
12    return ans;
13 }
14 ll f(ll N){
15     int k=0;
16     while(N){ // 把數字先分解到陣列
17         d[++k] = N%10;
18         N/=10;
19     }
20     return dfs(k,false,true);
21 }

```

6.15 質因數分解

```

1 LL func(const LL n,const LL mod,const int c)
2 {
3     return (LLmul(n,n,mod)+c+mod)%mod;
4 }

```

```

4 LL pollorrho(const LL n, const int c) { // 循
5     環長度
6     LL a=1, b=1;
7     a=func(a,n,c)%n;
8     b=func(b,n,c)%n; b=func(b,n,c)%n;
9     while(gcd(abs(a-b),n)!=1) {
10         a=func(a,n,c)%n;
11         b=func(b,n,c)%n; b=func(b,n,c)%n;
12     }
13     return gcd(abs(a-b),n);
14 }
15 void prefactor(LL &n, vector<LL> &v) {
16     for(int i=0;i<12;++i) {
17         while(n%prime[i]==0) {
18             v.push_back(prime[i]);
19             n/=prime[i];
20         }
21     }
22 }
23 void smallfactor(LL n, vector<LL> &v) {
24     if(n<MAXPRIME) {
25         while(isp[is8]) {
26             v.push_back(isp[is8]);
27             n/=isp[is8];
28         }
29         v.push_back(n);
30     } else {
31         for(int i=0;i<primecnt&&prime[i]*prime[i]
32             ]<=n;++i) {
33             while(n%prime[i]==0) {
34                 v.push_back(prime[i]);
35                 n/=prime[i];
36             }
37             if(n!=1) v.push_back(n);
38         }
39     }
40 }
41 void comfactor(const LL &n, vector<LL> &v) {
42     if(n<1e9) {
43         smallfactor(n,v);
44         return;
45     }
46     if(Isprime(n)) {
47         v.push_back(n);
48         return;
49     }
50     LL d;
51     for(int c=3; c<=n; c++) {
52         d = pollorrho(n,c);
53         if(d!=n) break;
54     }
55     comfactor(d,v);
56     comfactor(n/d,v);
57 }
58 void Factor(const LL &x, vector<LL> &v) {
59     LL n = x;
60     if(n==1) { puts("Factor 1"); return; }
61     prefactor(n,v);
62     if(n==1) return;
63     comfactor(n,v);

```

```

64     sort(v.begin(),v.end());
65 }
66 void AllFactor(const LL &n,vector<LL> &v) {
67     vector<LL> tmp;
68     Factor(n,tmp);
69     v.clear();
70     v.push_back(1);
71     int len;
72     LL now=1;
73     for(int i=0;i<tmp.size();++i) {
74         if(i==0 || tmp[i]!=tmp[i-1]) {
75             len = v.size();
76             now = 1;
77         }
78         now*=tmp[i];
79         for(int j=0;j<len;++j)
80             v.push_back(v[j]*now);
81     }
82 }

```

7 String

7.1 ac 自動機

```

1 int trie[(int)5e5 + 5][26], node_cnt = 0,
2 fail[(int)5e5 + 5]; // ac_automachine
3 int pattern_end[(int)5e5 + 5];
4 void insert(string s, int id)
5 {
6     int u = 0;
7     for(char c : s){
8         int v = c - 'a';
9         if(!trie[u][v]) trie[u][v] = ++
10             node_cnt;
11         u = trie[u][v];
12     }
13     pattern_end[id] = u;
14 }
15 void build_ac_automachine()
16 {
17     //bfs
18     queue<int> q;
19     for(int i = 0; i < 26; i++) if(trie[0][i]
20         ] > 0) q.push(trie[0][i]);
21     while(q.size()){
22         int x = q.front(); q.pop();
23         for(int i = 0; i < 26; i++){
24             if(trie[x][i] > 0){
25                 fail[trie[x][i]] = trie[fail
26                     [x]][i];
27                 q.push(trie[x][i]);
28             }
29             else trie[x][i] = trie[fail[x]][i];
30         }
31     }
32 }

```

7.2 KMP

```
1 vector<int> pi(s.size(), 0);
2 for(int i = 1, j = 0; i < s.size(); i++){
3     while(j > 0 and s[i] != s[j]) j = pi[j] - 1;
4     if(s[i] == s[j]) j++;
5     pi[i] = j;
6 }
```

7.3 manacher

```
1 string s, t = "^";
2 cin >> s;
3 for(char c : s){
4     t += "#";
5     t += c;
6 }
7 t += "##";
8 vector<int> p(t.size(), 0);
9 int c = 0, r = 0, len = 0, center = 0;
10 for(int i = 1; i < t.size() - 1; i++){
11     if(i < r) p[i] = min(r - i, p[2 * c - i]);
12     while(t[i + p[i] + 1] == t[i - p[i] - 1]) p[i]++;
13     if(i + p[i] > r){
14         r = i + p[i];
15         c = i;
16     }
17     if(p[i] > len){
18         len = p[i];
19         center = i;
20     }
21 }
22 cout << s.substr((center - len) / 2, len) <<
    "\n"; //Longest Palindrome
```

7.4 suffix-array

```
1 vector<int> buildSuffixArray(string s, int n)
2 {
3     vector<int> sa(n), tmp(n), rank(n);
4     /*
5     sa[i]: 從sa[i]開始的後綴字串是第i小
6     rank[i]: 從i開始的後綴字串是第幾小
7     tmp: rank的暫存
8     */
9     for(int i = 0; i < n; i++){ // 初始化長度為1的後綴字串
10         sa[i] = i;
11         rank[i] = s[i] - 'a';
12     }
13     for(int k = 1; k < n; k <= 1){
14         auto cmp = [&](int i, int j){ //比較長度為2*k的字串
```

```
15         if(rank[i] != rank[j]) return
16             rank[i] < rank[j];
17         int ri = i + k < n ? rank[i + k] : -1;
18         int rj = j + k < n ? rank[j + k] : -1;
19         return ri < rj;
20     };
21     sort(sa.begin(), sa.end(), cmp); //
22     /* sa 按照字典序(rank)排列
23     更新rank
24     要先用tmp存，因為cmp會用到rank的舊資料
25     */
26     tmp[sa[0]] = 0;
27     for(int i = 1; i < n; i++) tmp[sa[i]] = tmp[sa[i - 1]] + (cmp(sa[i - 1], sa[i]) ? 1 : 0);
28     rank = tmp;
29     return sa;
30 }
31 vector<int> buildLCP(string s, vector<int> sa, int n)
32 {
33     vector<int> rank(n, 0); // rank[i]: 從s[i]開始的字尾字串按照字典序排序後的位置
34     for(int i = 0; i < n; i++) rank[sa[i]] = i;
35     int h = 0; // 因為是按字串s原始位置順序計算，用前綴相似的特性，h每次最多只會減少1
36     vector<int> lcp(n, 0); // 與字典序前一個的字尾字串的最長共同前綴長度
37     for(int i = 0; i < n; i++){
38         if(rank[i] > 0){
39             int j = sa[rank[i] - 1]; // 字典序前一個的字尾字串
40             while(i + h < n and j + h < n and s[i + h] == s[j + h]) h++;
41             lcp[rank[i]] = h;
42             if(h > 0) h--;
43         }
44     }
45     return lcp;
46 }
47 int ans = 0, n = s.size();
48 /* ans: the number of distinct substrings that appear in a string
49 for(int i = 0; i < n; i++) ans += n - sa[i] - lcp[i];
50 */
51 (n - sa[i]): 這個字尾本來可以貢獻這麼多個字字串。
52 lcp[i]: 扣掉跟字典序前一個字尾長得一模一樣的前綴，因為那些已經算過了。
53 */
```

8 Tarjan

8.1 dominator tree

```
1 struct dominator_tree{
2     static const int MAXN=5005;
3     int n; // 1-base
4     vector<int> G[MAXN], rG[MAXN];
5     int pa[MAXN], dfn[MAXN], id[MAXN], dfnCnt;
6     int semi[MAXN], idom[MAXN], best[MAXN];
7     vector<int> tree[MAXN]; // tree here
8     void init(int _n){
9         n = _n;
10        for(int i=1; i<=n; ++i)
11            G[i].clear(), rG[i].clear();
12    }
13    void add_edge(int u, int v){
14        G[u].push_back(v);
15        rG[v].push_back(u);
16    }
17    void dfs(int u){
18        id[dfn[u]=++dfnCnt]=u;
19        for(auto v:G[u]) if(!dfn[v])
20            dfs(v), pa[dfn[v]]=dfn[u];
21    }
22    int find(int y, int x){
23        if(y <= x) return y;
24        int tmp = find(pa[y], x);
25        if(semi[best[y]] > semi[best[pa[y]]])
26            best[y] = best[pa[y]];
27        return pa[y] = tmp;
28    }
29    void tarjan(int root){
30        dfnCnt = 0;
31        for(int i=1; i<=n; ++i){
32            dfn[i] = idom[i] = 0;
33            tree[i].clear();
34            best[i] = semi[i] = i;
35        }
36        dfs(root);
37        for(int i=dfnCnt; i>1; --i){
38            int u = id[i];
39            for(auto v:rG[u]) if(v=dfn[v]){
40                find(v, i);
41                semi[i]=min(semi[i], semi[best[v]]);
42            }
43            tree[semi[i]].push_back(i);
44            for(auto v:tree[pa[i]]){
45                find(v, pa[i]);
46                idom[v] = semi[best[v]]==pa[i] ? pa[i] : best[v];
47            }
48            tree[pa[i]].clear();
49        }
50    }
51    for(int i=2; i<=dfnCnt; ++i){
52        if(idom[i] != semi[i])
53            idom[i] = idom[idom[i]];
54        tree[id[idom[i]]].push_back(id[i]);
55    }
56 }
57 } dom;
```

8.2 tnfshb017 2 sat

```
1 #include<bits/stdc++.h>
2 using namespace std;
3 #define MAXN 8001
4 #define MAXN2 MAXN*4
5 #define n(X) ((X)+2*N)
6 vector<int> v[MAXN2], rv[MAXN2], vis_t;
7 int N,M;
8 void addedge(int s,int e){
9     v[s].push_back(e);
10    rv[e].push_back(s);
11 }
12 int scc[MAXN2];
13 bool vis[MAXN2]={false};
14 void dfs(vector<int> *uv,int n,int k=-1){
15     vis[n]=true;
16     for(int i=0;i<uv[n].size();++i)
17         if(!vis[uv[n][i]])
18             dfs(uv,uv[n][i],k);
19     if(uv==v)vis_t.push_back(n);
20     scc[n]=k;
21 }
22 void solve(){
23     for(int i=1;i<=N;++i){
24         if(!vis[i])dfs(v,i);
25         if(!vis[n(i)])dfs(v,n(i));
26     }
27     memset(vis,0,sizeof(vis));
28     int c=0;
29     for(int i=vis_t.size()-1;i>=0;--i)
30         if(!vis[vis_t[i]])
31             dfs(rv,vis_t[i],c++);
32 }
33 int main(){
34     int a,b;
35     scanf("%d%d",&N,&M);
36     for(int i=1;i<=N;++i){
37         // (A or B)&(!A & !B) A^B
38         a=i*2-1;
39         b=i*2;
40         addedge(n(a),b);
41         addedge(n(b),a);
42         addedge(a,n(b));
43         addedge(b,n(a));
44     }
45     while(M--){
46         scanf("%d%d",&a,&b);
47         a = a>0?a*2-1:-a*2;
48         b = b>0?b*2-1:-b*2;
49         // A or B
50         addedge(n(a),b);
51         addedge(n(b),a);
52     }
53     solve();
54     bool check=true;
55     for(int i=1;i<=2*N;++i)
56         if(scc[i]==scc[n(i)])
57             check=false;
58     if(check){
59         printf("%d\n",N);
60         for(int i=1;i<=2*N;i+=2){
61             if(scc[i]>scc[i+2*N]) putchar('+');
62             else putchar('-');
63         }
64     }
```

```

64     puts("");
65 }else puts("0");
66 return 0;
67 }

```

8.3 橋連通分量

```

1 #define N 1005
2 struct edge{
3     int u,v;
4     bool is_bridge;
5     edge(int u=0,int v=0):u(u),v(v),is_bridge(0){}
6 };
7 vector<edge> E;
8 vector<int> G[N]; // 1-base
9 int low[N],vis[N],Time;
10 int bcc_id[N],bridge_cnt,bcc_cnt; // 1-base
11 int st[N],top; // BCC用
12 void add_edge(int u,int v){
13     G[u].push_back(E.size());
14     E.emplace_back(u,v);
15     G[v].push_back(E.size());
16     E.emplace_back(v,u);
17 }
18 void dfs(int u,int re=-1){ // u當前點, re為u連
    接前一個點的邊
19     int v;
20     low[u]=vis[u]=++Time;
21     st[top++]=u;
22     for(int e:G[u]){
23         v=E[e].v;
24         if(!vis[v]){
25             dfs(v,e^1); // e^1 反向邊
26             low[u]=min(low[u],low[v]);
27             if(vis[u]<low[v]){
28                 E[e].is_bridge=E[e^1].is_bridge=1;
29                 ++bridge_cnt;
30             }
31         }else if(vis[v]<vis[u]&&e!=re){
32             low[u]=min(low[u],vis[v]);
33         }
34     }
35     if(vis[u]==low[u]){ // 處理BCC
36         ++bcc_cnt; // 1-base
37         do bcc_id[v=st[--top]]=bcc_cnt; // 每個點
            所在的BCC
38         while(v!=u);
39     }
40     void bcc_init(int n){
41         Time=bcc_cnt=bridge_cnt=top=0;
42         E.clear();
43         for(int i=1;i<=n;++i){
44             G[i].clear();
45             vis[i]=bcc_id[i]=0;
46         }
47     }

```

8.4 雙連通分量 & 割點

```

1 #define N 1005
2 vector<int> G[N]; // 1-base
3 vector<int> bcc[N]; // 存每塊雙連通分量的點
4 int low[N],vis[N],Time;
5 int bcc_id[N],bcc_cnt; // 1-base
6 bool is_cut[N]; // 是否為割點
7 int st[N],top;
8 void dfs(int u,int pa=-1){ // u當前點, pa父親
9     int t, child=0;
10     low[u]=vis[u]=++Time;
11     st[top++]=u;
12     for(int v:G[u]){
13         if(!vis[v]){
14             dfs(v,u), ++child;
15             low[u]=min(low[u],low[v]);
16             if(vis[u]<=low[v]){
17                 is_cut[u]=1;
18                 bcc[++bcc_cnt].clear();
19                 do{
20                     bcc_id[t=st[--top]]=bcc_cnt;
21                     bcc[bcc_cnt].push_back(t);
22                 }while(t!=v);
23                 bcc_id[u]=bcc_cnt;
24                 bcc[bcc_cnt].push_back(u);
25             }
26             }else if(vis[v]<vis[u]&&v!=pa) // 反向邊
27                 low[u] = min(low[u],vis[v]);
28         } // u是dfs樹的根要特判
29         if(pa!=-1&&child<2) is_cut[u]=0;
30     }
31     void bcc_init(int n){
32         Time=bcc_cnt=top=0;
33         for(int i=1;i<=n;++i){
34             G[i].clear();
35             is_cut[i]=vis[i]=bcc_id[i]=0;
36         }
37     }

```

9 compare

9.1 check

```

1 #include<bits/stdc++.h>
2 using namespace std;
3 int main() {
4     // 1. 在迴圈外編譯, 並檢查是否成功
5     cout << "Compiling files..." << endl;
6
7     // system() 回傳 0 代表成功執行
8     int c1 = system("g++ -Wall -std=c++20
9         sol.cpp -o sol.exe");
10    int c2 = system("g++ -Wall -std=c++20 bf
11        .cpp -o bf.exe");
12    int c3 = system("g++ -Wall -std=c++20
13        gen.cpp -o gen.exe");
14
15    if (c1 != 0 || c2 != 0 || c3 != 0) {
16        cout << "Compilation Failed! Check
17            if sol.cpp, bf.cpp, and gen.cpp
18            exist." << endl;
19    }

```

```

14     return 1;
15 }
16
17 cout << "Compilation successful.
18     Starting stress test..." << endl;
19
20 for (int T = 1; ; T++) {
21     // 2. 執行程序
22     system("gen.exe > input.txt");
23     system("sol.exe < input.txt > sol.
24         out");
25     system("bf.exe < input.txt > bf.out"
26         );
27
28     // 3. 使用 fc /W 比對
29     if (system("fc sol.out bf.out > nul"
30         )) {
31         system("fc sol.out bf.out");
32         cout << "WA on Test #" << T << "
33             !" << endl;
34         // 發生錯誤時停止, 此時 input.
35         txt 就是出錯的測資
36         break;
37     } else {
38         if (T % 10 == 0) cout << "Passed
39             " << T << " tests..." <<
40             endl;
41     }
42 }
43
44 return 0;
45 }

```

9.2 gen

```

1 #include<bits/stdc++.h>
2 using namespace std;
3
4 // 使用隨機種子, 確保每次生成的數據都不同
5 mt19937_64 rng(chrono::steady_clock::now().
6     time_since_epoch().count());
7
8 // 生成 [L, r] 範圍內的隨機整數
9 long long rnd(long long l, long long r) {
10     return uniform_int_distribution<long
11         long>(l, r)(rng);
12 }
13
14 // 生成一棵隨機樹 (N 個點, N-1 條邊)
15 void gen_tree(int n) {
16     for (int i = 2; i <= n; i++) {
17         // 讓每個點 i 連向 [1, i-1] 中的隨機
18         點
19         int u = i;
20         int v = rnd(1, i - 1);
21         cout << u << " " << v << "\n";
22     }
23 }

```

10 default

10.1 debug

```

1 #ifdef DEBUG
2 #define dbg(...) {\
3     fprintf(stderr,"%s - %d : (%s) = ",
4         __PRETTY_FUNCTION__, __LINE__, #
5         __VA_ARGS__); \
6     _DO(__VA_ARGS__); \
7 }
8 template<typename I> void _DO(I&&x){cerr<<x
9     <<endl;}
10 template<typename I, typename...T> void _DO(I
11     &&x,T&&...tail){cerr<<x<<" "; _DO(tail
12     ...);}
13 #else
14 #define dbg(...)
15 #endif

```

10.2 ext

```

1 #include<bits/extc++.h>
2 #include<ext/pd_ds/assoc_container.hpp>
3 #include<ext/pd_ds/tree_policy.hpp>
4 using namespace __gnu_cxx;
5 using namespace __gnu_pbds;
6 template<typename T>
7 using pbds_set = tree<T,null_type,less<T>,
8     rb_tree_tag,
9     tree_order_statistics_node_update>;
10 template<typename T, typename U>
11 using pbds_map = tree<T,U,less<T>,
12     rb_tree_tag,
13     tree_order_statistics_node_update>;
14 using heap = __gnu_pbds::priority_queue<int>;
15 //s.find_by_order(1); // 0 base
16 //s.order_of_key(1);

```

10.3 IncStack

```

1 //Magic
2 #pragma GCC optimize "Ofast"
3 //stack resize, change esp to rsp if 64-bit
4 system
5 asm("mov %0,%esp\n" :: "g"(mem+10000000));
6 -Wl,--stack,214748364 -trigraphs
7 #pragma comment(linker, "/STACK
8     :1024000000,1024000000")
9 //Linux stack resize
10 #include<sys/resource.h>
11 void increase_stack(){
12     const rlim_t ks=64*1024*1024;
13     struct rlimit rl;
14     int res=getrlimit(RLIMIT_STACK,&rl);
15     if(!res&&rl.rlim_cur<ks){
16         rl.rlim_cur=ks;
17     }

```

```

15     res=setrlimit(RLIMIT_STACK,&rl);
16 }
17 }

```

10.4 input

```

1 inline int read(){
2     int x=0; bool f=0; char c=getchar();
3     while(ch<'0' || '9'<ch)f|=ch=='-',ch=getchar();
4     while('0'<=ch&&ch<='9')x=x*10-'0'+ch,ch=getchar();
5     return f?-x:x;
6 }
7 // #!/bin/bash
8 // g++ -std=c++11 -O2 -Wall -Wextra -Wno-unused-result -DDEBUG $1 && ./a.out
9 // -fsanitize=address -fsanitize=undefined -fsanitize=return

```

11 other

11.1 WhatDay

```

1 int whatday(int y,int m,int d){
2     if(m<=2)m+=12,--y;
3     if(y<1752||y==1752&&m<9||y==1752&&m==9&&d<3)
4         return (d+2*m+3*(m+1)/5+y+y/4+5)%7;
5     return (d+2*m+3*(m+1)/5+y+y/4-y/100+y/400)%7;
6 }

```

11.2 上下最大正方形

```

1 void solve(int n,int a[],int b[]){// 1-base
2     int ans=0;
3     deque<int>da,db;
4     for(int l=1,r=1;r<=n;++r){
5         while(da.size()&&a[da.back()]>=a[r]){
6             da.pop_back();
7         }
8         da.push_back(r);
9         while(db.size()&&b[db.back()]>=b[r]){
10             db.pop_back();
11         }
12         db.push_back(r);
13         for(int d=a[da.front()]+b[db.front()];r-l+1>d;++l){
14             if(da.front()==l)da.pop_front();
15             if(db.front()==l)db.pop_front();
16             if(da.size()&&db.size()){
17                 d=a[da.front()]+b[db.front()];
18             }
19         }
20     }

```

```

20     ans=max(ans,r-l+1);
21 }
22 printf("%d\n",ans);
23 }

```

11.3 最大矩形

```

1 LL max_rectangle(vector<int> s){
2     stack<pair<int,int>> st;
3     st.push(make_pair(-1,0));
4     s.push_back(0);
5     LL ans=0;
6     for(size_t i=0;i<s.size();++i){
7         int h=s[i];
8         pair<int,int> now=make_pair(h,i);
9         while(h<st.top().first){
10             now=st.top();
11             st.pop();
12             ans=max(ans,(LL)(i-now.second)*now.first);
13         }
14         if(h>st.top().first){
15             st.push(make_pair(h,now.second));
16         }
17     }
18     return ans;
19 }

```

12 other language

12.1 java

12.1.1 文件操作

```

1 import java.io.*;
2 import java.util.*;
3 import java.math.*;
4 import java.text.*;
5
6 public class Main{
7
8     public static void main(String args[]){
9         throws FileNotFoundException,
10         IOException
11         Scanner sc = new Scanner(new FileReader(
12             "a.in"));
13         PrintWriter pw = new PrintWriter(new
14             FileWriter("a.out"));
15         int n,m;
16         n=sc.nextInt();//读入下一个INT
17         m=sc.nextInt();
18
19         for(ci=1; ci<=c; ++ci){
20             pw.println("Case #"+ci+": easy for
21                 output");
22         }
23     }
24 }

```

```

19     pw.close();//关闭流并释放，这个很重要，
20     否则是没有输出的
21     sc.close();//关闭流并释放
22 }

```

12.1.2 优先队列

```

1 PriorityQueue queue = new PriorityQueue(1,
2     new Comparator(){
3         public int compare(Point a, Point b){
4             if(a.x < b.x || a.x == b.x && a.y < b.y)
5                 return -1;
6             else if(a.x == b.x && a.y == b.y)
7                 return 0;
8             else return 1;
9         }
10 });

```

12.1.3 Map

```

1 Map map = new HashMap();
2 map.put("sa","dd");
3 String str = map.get("sa").toString();
4
5 for(Object obj : map.keySet()){
6     Object value = map.get(obj);
7 }

```

12.1.4 sort

```

1 static class cmp implements Comparator{
2     public int compare(Object o1,Object o2){
3         BigInteger b1=(BigInteger)o1;
4         BigInteger b2=(BigInteger)o2;
5         return b1.compareTo(b2);
6     }
7 }
8 public static void main(String[] args)
9     throws IOException{
10     Scanner cin = new Scanner(System.in);
11     int n;
12     n=cin.nextInt();
13     BigInteger[] seg = new BigInteger[n];
14     for(int i=0;i<n;i++){
15         seg[i]=cin.nextBigInteger();
16         Arrays.sort(seg,new cmp());
17     }

```

12.2 python heap

```

1 import heapq
2
3 heap = [7,1,2,2]
4 heapq.heapify(heap)
5 print(heap) # [1, 2, 2, 7]
6 heapq.heappush(heap, 5)
7 print(heap) # [1, 2, 2, 7, 5]
8 print(heapq.heappop(heap)) # 1
9 print(heap) # [2, 2, 5, 7]

```

12.3 python input

```

1 ans = sum(map(float, input().split()))
2 # input: 1.1 2.2 3.3 4.4 5.5
3 print(ans) # 16.5
4
5 (n, m) = map(int, input().split()) # 300 200
6 print(n * m) # 60000
7
8 Arr = list(map(int, input().split()))
9 # input: 1 2 3 4 5
10 print(Arr) # [1, 2, 3, 4, 5]

```

12.4 python output

```

1 hello = 'Hello'
2 world = 7122
3 print(f'{hello} {world}') # Hello 7122
4
5 import math
6 print(f'PI is approximately {math.pi:.3f}.')
7 # PI is approximately 3.142.
8
9 print('AAA {} BBB {}'.format('Jin', 'KeLa'))
10 # AAA Jin BBB "KeLa!"
11
12 hello = 'hello, world\n'
13 hellos = repr(hello)
14 print(hellos) # 'hello, world\n'
15
16 x = 32.5
17 y = 40000
18 print(repr((x, y, ('spam', 'eggs'))))
19 # "(32.5, 40000, ('spam', 'eggs'))"
20
21 x = 7
22 print(eval('3 * x')) # 21

```

13 zformula

13.1 formula

13.1.1 Pick 公式

給定頂點坐標均是整點的簡單多邊形，面積 = 內部格點數 + 邊上格點數/2-1

13.1.2 圖論

- 對於平面圖 $\cdot F = E - V + C + 1$ $\cdot C$ 是連通分量數
- 對於平面圖 $\cdot E \leq 3V - 6$
- 對於連通圖 G $\cdot$ 最大獨立點集的大小設為 $I(G)$ $\cdot$ 最大匹配大小設為 $M(G)$ $\cdot$ 最小點覆蓋設為 $Cv(G)$ $\cdot$ 最小邊覆蓋設為 $Ce(G)$ $\cdot$ 對於任意連通圖：

$$(a) \ I(G) + Cv(G) = |V|$$

$$(b) \ M(G) + Ce(G) = |V|$$

- 對於連通二分圖：

$$(a) \ I(G) = Cv(G)$$

$$(b) \ M(G) = Ce(G)$$

- 最大權閉合圖：

$$(a) \ C(u, v) = \infty, (u, v) \in E$$

$$(b) \ C(v, v) = W_v, W_v > 0$$

$$(c) \ C(v, T) = -W_v, W_v < 0$$

$$(d) \ ans = \sum_{W_v > 0} W_v - flow(S, T)$$

- 最大密度子圖：

$$(a) \ 求 \ max \left(\frac{W_e + W_v}{|V|} \right), e \in E', v \in V'$$

$$(b) \ U = \sum_{v \in V} 2W_v + \sum_{e \in E} W_e$$

$$(c) \ C(u, v) = W_{(u, v)}, (u, v) \in E \cdot \text{雙向邊}$$

$$(d) \ C(S, v) = U, v \in V$$

$$(e) \ D_u = \sum_{(u, v) \in E} W_{(u, v)}$$

$$(f) \ C(v, T) = U + 2g - D_v - 2W_v, v \in V$$

$$(g) \ 二分搜 \ g:$$

$$l = 0, r = U, eps = 1/n^2$$

$$\text{if}((U \times |V| - flow(S, T))/2 > 0) \ l = mid$$

$$\text{else } r = mid$$

$$(h) \ ans = min_cut(S, T)$$

$$(i) \ |E| = 0 \text{ 要特殊判斷}$$

- 弦圖：

- 點數大於 3 的環都要有一條弦
- 完美消除序列從後往前依次給每個點染色，給每個點染上可以染的最小顏色
- 最大團大小 = 色數
- 最大獨立集: 完美消除序列從前往後能選就選
- 最小團覆蓋: 最大獨立集的點和他延伸的邊構成
- 區間圖是弦圖
- 區間圖的完美消除序列: 將區間按造又端點由小到大的排序
- 區間圖染色: 用線段樹做

13.1.3 dinic 特殊圖複雜度

- 單位流： $O\left(\min\left(V^{3/2}, E^{1/2}\right)E\right)$
- 二分圖： $O\left(V^{1/2}E\right)$

13.1.4 0-1 分數規劃

$x_i = \{0, 1\} \cdot x_i$ 可能會有其他限制，求 $max\left(\frac{\sum B_i x_i}{\sum C_i x_i}\right)$

- $D(i, g) = B_i - g \times C_i$
- $f(g) = \sum D(i, g) x_i$
- $f(g) = 0$ 時 g 為最佳解， $f(g) < 0$ 沒有意義
- 因為 $f(g)$ 單調可以二分搜 g
- 或用 Dinkelbach 通常比較快

```
1 binary_search(){
2   while(r-l>eps){
3     g=(l+r)/2;
4     for(i:所有元素)D[i]=B[i]-g*C[i]; //D(i,g)
5     找出一組合法x[i]使f(g)最大;
6     if(f(g)>0) l=g;
7     else r=g;
8   }
9   Ans = r;
10 }
11 Dinkelbach(){
12   g=任意狀態(通常設為0);
13   do{
14     Ans=g;
15     for(i:所有元素)D[i]=B[i]-g*C[i]; //D(i,g)
16     找出一組合法x[i]使f(g)最大;
17     p=0, q=0;
18     for(i:所有元素)
19       if(x[i]) p+=B[i], q+=C[i];
20     g=p/q; //更新解，注意q=0的情況
21   }while(abs(Ans-g)>EPS);
22   return Ans;
23 }
```

13.1.5 學長公式

- $\sum_{d|n} \phi(n) = n$
- $g(n) = \sum_{d|n} f(d) \Rightarrow f(n) = \sum_{d|n} \mu(d) \times g(n/d)$
- Harmonic series $H_n = \ln(n) + \gamma + 1/(2n) - 1/(12n^2) + 1/(120n^4)$
- $\gamma = 0.57721566490153286060651209008240243104215$
- 格雷碼 = $n \oplus (n >> 1)$
- $SG(A + B) = SG(A) \oplus SG(B)$
- 選轉矩陣 $M(\theta) = \begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix}$

13.1.6 基本數論

- $\sum_{d|n} \mu(n) = [n == 1]$
- $g(m) = \sum_{d|m} f(d) \Leftrightarrow f(m) = \sum_{d|m} \mu(d) \times g(m/d)$
- $\sum_{i=1}^n \sum_{j=1}^m \text{互質數量} = \sum \mu(d) \lfloor \frac{n}{d} \rfloor \lfloor \frac{m}{d} \rfloor$
- $\sum_{i=1}^n \sum_{j=1}^n lcm(i, j) = n \sum_{d|n} d \times \phi(d)$

13.1.7 排組公式

- k 卡特蘭 $\frac{C_n^{kn}}{n(k-1)+1} \cdot C_m^n = \frac{n!}{m!(n-m)!}$
- $H(n, m) \cong x_1 + x_2 \dots + x_n = k, num = C_k^{n+k-1}$
- Stirling number of $2^{nd}, n$ 人分 k 組方法數目
 - $S(0, 0) = S(n, n) = 1$
 - $S(n, 0) = 0$
 - $S(n, k) = kS(n-1, k) + S(n-1, k-1)$

- Bell number, n 人分任意多組方法數目

- $B_0 = 1$
- $B_n = \sum_{i=0}^n S(n, i)$
- $B_{n+1} = \sum_{k=0}^n C_k^n B_k$
- $B_{p+n} \equiv B_n + B_{n+1} \pmod{p}$, p is prime
- $B_{p^m+n} \equiv mB_n + B_{n+1} \pmod{p}$, p is prime
- From $B_0 : 1, 1, 2, 5, 15, 52, 203, 877, 4140, 21147, 115975$

- Derangement, 錯排, 沒有人在自己位置上

- $D_n = n!(1 - \frac{1}{1!} + \frac{1}{2!} - \frac{1}{3!} \dots + (-1)^n \frac{1}{n!})$
- $D_n = (n-1)(D_{n-1} + D_{n-2}), D_0 = 1, D_1 = 0$
- From $D_0 : 1, 0, 1, 2, 9, 44, 265, 1854, 14833, 133496$

- Binomial Equality

- $\sum_k \binom{r}{m+k} \binom{s}{n-k} = \binom{r+s}{l+m+n}$
- $\sum_k \binom{m+k}{l} \binom{s}{n-k} = \binom{l+s}{l-m+n}$
- $\sum_k \binom{l}{m+k} \binom{s+k}{n} (-1)^k = (-1)^{l+m} \binom{s-m}{n-l}$
- $\sum_{k \leq l} \binom{l-k}{m} \binom{s}{k-n} (-1)^k = (-1)^{l+m} \binom{s-m-1}{l-k-n}$
- $\sum_{0 \leq k \leq l} \binom{l-k}{m} \binom{q+k}{n} = \binom{l+q+1}{m+n+1}$
- $\binom{r}{k} = (-1)^k \binom{k-r-1}{k}$
- $\binom{r}{m} \binom{m}{k} = \binom{r}{k} \binom{r-k}{m-k}$
- $\sum_{k \leq n} \binom{r+k}{k} = \binom{r+n+1}{m+1}$
- $\sum_{0 \leq k \leq n} \binom{k}{m} = \binom{n+1}{m+1}$
- $\sum_{k \leq m} \binom{m+r}{k} x^k y^{m-k} = \sum_{k \leq m} \binom{-r}{k} (-x)^k (x+y)^{m-k}$

13.1.8 冪次, 冪次和

- $a^{b \% p} P = a^{b \% \varphi(p) + \varphi(p)}, b \geq \varphi(p)$
- $1^3 + 2^3 + 3^3 + \dots + n^3 = \frac{n^4}{4} + \frac{n^3}{2} + \frac{n^2}{4}$
- $1^4 + 2^4 + 3^4 + \dots + n^4 = \frac{n^5}{5} + \frac{n^4}{2} + \frac{n^3}{3} - \frac{n}{30}$
- $1^5 + 2^5 + 3^5 + \dots + n^5 = \frac{n^6}{6} + \frac{n^5}{2} + \frac{5n^4}{12} - \frac{n^2}{12}$
- $0^k + 1^k + 2^k + \dots + n^k = P(k), P(k) = \frac{(n+1)^{k+1} - \sum_{i=0}^{k-1} C_i^{k+1} P(i)}{k+1}, P(0) = n+1$
- $\sum_{k=0}^{m-1} k^n = \frac{1}{n+1} \sum_{k=0}^n C_k^{n+1} B_k m^{n+1-k}$
- $\sum_{j=0}^m C_j^{m+1} B_j = 0, B_0 = 1$
- 除了 $B_1 = -1/2$ $\cdot$ 剩下的奇數項都是 0
- $B_2 = 1/6, B_4 = -1/30, B_6 = 1/42, B_8 = -1/30, B_{10} = 5/66, B_{12} = -691/2730, B_{14} = 7/6, B_{16} = -3617/510, B_{18} = 43867/798, B_{20} = -174611/330,$

13.1.9 Burnside's lemma

- $|X/G| = \frac{1}{|G|} \sum_{g \in G} |X^g|$
- $X^g = t^{c(g)}$
- G 表示有幾種轉法， X^g 表示在那種轉法下，有幾種是會保持對稱的， t 是顏色數， $c(g)$ 是循環節不動的面數。
- 正立方體塗三顏色，轉 0 有 3^6 個元素不變，轉 90 有 6 種，每種有 3^3 不變，180 有 $3 \times 3^4 \cdot 120(\text{角})$ 有 $8 \times 3^2 \cdot 180(\text{邊})$ 有 $6 \times 3^3 \cdot \text{全部}$ $\frac{1}{24} (3^6 + 6 \times 3^3 + 3 \times 3^4 + 8 \times 3^2 + 6 \times 3^3) = \frac{57}{57}$

13.1.10 Count on a tree

$$1. \text{ Rooted tree: } s_{n+1} = \frac{1}{n} \sum_{i=1}^n (i \times a_i \times \sum_{j=1}^{\lfloor n/i \rfloor} a_{n+1-i \times j})$$

- Unrooted tree:

$$(a) \text{ Odd: } a_n - \sum_{i=1}^{n/2} a_i a_{n-i}$$

$$(b) \text{ Even: } Odd + \frac{1}{2} a_{n/2} (a_{n/2} + 1)$$

- Spanning Tree

$$(a) \text{ 完全圖 } n^n - 2$$

$$(b) \text{ 一般圖 (Kirchhoff's theorem) } M[i][i] = degree(V_i), M[i][j] = -1, \text{ if have } E(i, j), 0 \text{ if no edge. delete any one row and col in } A, ans = det(A)$$

```

21 //
22 //
23 //
24 //
25 //
26 //
27 //
28 //
29 //
30 //
31 //
32 //
33 //
34 // ~~~~~
35 // 佛祖保佑 永無BUG
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54 #
55 #
56 #
57 #
58 #
59 #
60 #
61 #
62 #
63 #
64 #
65 #
66 #
67 #
68 #
69 #
70 #
71 #
72 #
73 #
74 # 神獸保佑 永無BUG!
75
76
77 // ## #####
78 // ## ##
79 // ## ##
80 // ## ##
81 // ## ##

```

```

82 // ##
83 // ##
84 // ##
85 // #####
86 // ##
87 // ##
88 // ##
89 // ##
90 // ##
91 // ##
92 // ##
93 // #####
94 //
95 // 元首保佑 永無BUG
96
97 //
98 //
99 // < @ \
100 // \(\_)*\*****/\(_)/
101 // \(_)*\*****/\(_)/
102 // u/u/u/****\u\u\u
103 // u/u/****\u\u
104 // |*|*|
105 // | |
106 // /+--+ \
107 // || 卐 ||
108 // \|=//
109 //
110 // 神獸保佑 永無BUG

```

ACM ICPC Team Reference - CCU- oshiete536senpai

Contents

1 Computational Geometry	1	4.1 bellman-ford	2	7.3 manacher	6	12.1.1 文件操作	8
1.1 convex-hull	1	4.2 SCC	2	7.4 suffix-array	6	12.1.2 优先队列	8
2 Data Structure	1	5 Linear Programming	2	8 Tarjan	6	12.1.3 Map	8
2.1 segment tree lazytag	1	5.1 simplex	2	8.1 dominator tree	6	12.1.4 sort	8
2.2 treap	1	6 Number Theory	2	8.2 tnfsb017 2 sat	6	12.2 python heap	8
3 Flow	1	6.1 basic	2	8.3 橋連通分量	7	12.3 python input	8
3.1 dinic	1	6.2 bit set	3	8.4 雙連通分量 & 割點	7	12.4 python output	8
4 Graph	2	6.3 cantor expansion	3	9 compare	7	13 zformula	9
		6.4 FFT	3	9.1 check	7	13.1 formula	9
		6.5 find real root	3	9.2 gen	7	13.1.1 Pick 公式	9
		6.6 FWT	3	10 default	7	13.1.2 圖論	9
		6.7 LinearCongruence	4	10.1 debug	7	13.1.3 dinic 特殊圖複雜度	9
		6.8 Lucas	4	10.2 ext	7	13.1.4 0-1 分數規劃	9
		6.9 Matrix	4	10.3 IncStack	7	13.1.5 學長公式	9
		6.10 MillerRobin	4	10.4 input	8	13.1.6 基本數論	9
		6.11 NTT	4	11 other	8	13.1.7 排組公式	9
		6.12 Simpson	5	11.1 WhatDay	8	13.1.8 冪次, 冪次和	9
		6.13 外星模運算	5	11.2 上下最大正方形	8	13.1.9 Burnside's lemma	9
		6.14 數位統計	5	11.3 最大矩形	8	13.1.10 Count on a tree	9
		6.15 質因數分解	5	12 other language	8	14 Интернационал	10
		7 String	5	12.1 java	8	14.1 ganadoQuote	10
		7.1 ac 自動機	5			14.2 Интернационал	10
		7.2 KMP	6			14.3 保佑	10

ACM ICPC Judge Test - CCU- oshiete536senpai

C++ Resource Test

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 namespace system_test {
5
6 const size_t KB = 1024;
7 const size_t MB = KB * 1024;
8 const size_t GB = MB * 1024;

```

```

9 size_t block_size, bound;
10 void stack_size_dfs(size_t depth = 1) {
11     if (depth >= bound)
12         return;
13     int8_t ptr[block_size]; // 若無法編譯將
14                             // block_size 改成常數
15     memset(ptr, 'a', block_size);
16     cout << depth << endl;
17     stack_size_dfs(depth + 1);
18 }
19
20 void stack_size_and_runtime_error(size_t
21     block_size, size_t bound = 1024) {
22     system_test::block_size = block_size;
23     system_test::bound = bound;
24     stack_size_dfs();
25 }
26
27 double speed(int iter_num) {
28     const int block_size = 1024;
29     volatile int A[block_size];
30     auto begin = chrono::high_resolution_clock
31         ::now();
32     while (iter_num--)
33         for (int j = 0; j < block_size; ++j)
34             A[j] += j;
35     auto end = chrono::high_resolution_clock::
36         now();

```

```

37 chrono::duration<double> diff = end -
38     begin;
39 return diff.count();
40 }
41
42 void runtime_error_1() {
43     // Segmentation fault
44     int *ptr = nullptr;
45     *(ptr + 7122) = 7122;
46 }
47
48 void runtime_error_2() {
49     // Segmentation fault
50     int *ptr = (int *)memset;
51     *ptr = 7122;
52 }
53
54 void runtime_error_3() {
55     // munmap_chunk(): invalid pointer
56     int *ptr = (int *)memset;
57     delete ptr;
58 }
59
60 void runtime_error_4() {
61     // free(): invalid pointer
62     int *ptr = new int[7122];
63     ptr += 1;
64     delete[] ptr;
65 }

```

```

62
63 void runtime_error_5() {
64     // maybe illegal instruction
65     int a = 7122, b = 0;
66     cout << (a / b) << endl;
67 }
68
69 void runtime_error_6() {
70     // floating point exception
71     volatile int a = 7122, b = 0;
72     cout << (a / b) << endl;
73 }
74
75 void runtime_error_7() {
76     // call to abort.
77     assert(false);
78 }
79
80 } // namespace system_test
81
82 #include <sys/resource.h>
83 void print_stack_limit() { // only work in
84     Linux
85     struct rlimit l;
86     getrlimit(RLIMIT_STACK, &l);
87     cout << "stack_size = " << l.rlim_cur << "
88         byte" << endl;
89 }

```